Machine Learning

Lecture 5: Learning by Combining Models

A textbook treatment

Based on the lecture notes and slides of Mounir Ghogho

October 25, 2026

Contents

How to read this book **2**

5 Learning by Combining Models **3**
How to read the formulas in this chapter . . . 3
5.1 Why Combine Models? . . . 4
5.2 Averaging and Voting . . . 5
5.3 Decision Trees as Base Learners . . . 8
5.4 Bagging and Out-of-Bag Evaluation . . . 10
5.5 Random Forests . . . 12
5.6 Feature Importance . . . 14
5.7 Boosting for Regression . . . 16
5.8 Boosting for Binary Classification . . . 19
5.9 XGBoost: One Objective for Structure and Leaf Values . . . 22
5.10 Multi-Class Boosting and Other Libraries . . . 25
5.11 Bagging versus Boosting, and When Ensembles Do Not Help . . . 26
5.12 Stacking . . . 27
5.13 Mixture of Experts . . . 29
5.14 Where Ensembles Are Used, and Lab 3 . . . 30
Notes on the original slides . . . 30
Chapter takeaways . . . 31

How to read this book

Each concept in this book is presented in the same rhythm.

1. **Motivation and intuition.** Why do we care? What problem does the idea solve, and what picture should you hold in your head?
2. **Formal statement.** The precise definition or theorem, set in a coloured box so it is easy to find when revising.
3. **Proof.** Every claim is proved. Longer proofs are split into labelled steps, and the one idea that makes the proof work is isolated in a *Key insight* box.
4. **Interpretation and discussion.** What the result really says, where it applies, and which misreadings to avoid.
5. **Examples.** From "trivially small" to "genuinely useful."
6. **Techniques, classic problems and gotchas.** Computational shortcuts, proof patterns, exam favourites with one-line solutions, and the mistakes everyone makes once.

Margin notes carry the informal, "say it out loud" version of what is happening in the main text. Every major section opens with a *What are we trying to get?* box that states the goal in one breath before any mathematics, and closes with a summary box.

Prerequisites. Basic probability (random variables, conditional distributions, expectations), vectors in $\mathbb{R}^d$, and a first course in calculus. The information-theoretic quantities used (entropy, cross-entropy, Kullback–Leibler divergence) are defined and proved from scratch where they appear.

Chapter 5

Learning by Combining Models

How to read the formulas in this chapter

Chapters 3 and 4 dealt with *one* fitted model: its bias and variance, its posterior estimate, its validation. This chapter fits *many* models and combines them. The new symbols are listed here; as before, every formula is followed by a *Reading the formula* box.

Symbol	Meaning in words
M	number of fitted models (trees, classifiers) in the ensemble
$\widehat{f}_m(x)$, $\widehat{p}_m(y = c \mid x)$	prediction, or posterior estimate, of the m-th fitted model
$\widehat{f}_{\text{ens}}(x)$	the ensemble prediction (average or vote)
σ^2, ρ	error variance of one model; correlation between the errors of two models
$\mathcal{D}^*_m$	the m-th *bootstrap sample*: N draws with replacement from $\mathcal{D}$
$\text{count}(i)$	how many times observation i appears in one bootstrap sample
$\mathcal{M}_i$	the *out-of-bag* models for observation i: those whose bootstrap sample omits i
$S_1, \ldots, S_J$; θ_j	the regions (leaves) of a tree and the constant predicted in each
$I(S)$	impurity of a set of observations (Gini for classification, squared error for regression)
$\widehat{p}_c(S)$	fraction of observations in S with class c
m_{try}	number of randomly chosen candidate features offered at each split of a random forest
Δ_t, MDI_j, I_j	credit of split t; mean-decrease-in-impurity and permutation importance of feature j
$F_m(x)$, $T_m(x)$, η	boosted ensemble after m rounds; the m-th tree; the learning rate (shrinkage)
$r_m^{(i)}$	residual, or pseudo-residual, of observation i before round m
$w_{q,m}$, $S_{q,m}$, Q_m	output of leaf q of tree m; its region; number of leaves of tree m
$g^{(i)}$, $h^{(i)}$	first and second derivative of the loss of observation i with respect to its current logit
G_q, H_q	sums of $g^{(i)}$ and $h^{(i)}$ over the observations in leaf q
λ, γ	XGBoost penalties: on squared leaf outputs; per leaf
z_m, $h_{\hat{\gamma}}$	stacking: base-model predictions and the fitted meta-model
$g_m(x)$, $a_m(x)$	mixture of experts: gate weight and gate score of expert m

5.1 Why Combine Models?

What are we trying to get?

Chapter 3 showed that a flexible model trained on one sample has high *variance*: a different sample would have produced a noticeably different fit. We want to see why *averaging several fits* attacks exactly this problem, under what condition it works (their errors must differ), and which three ways of producing different fits are in common use.

The wisdom of the crowd

Many rough guesses, averaged, can beat each rough guess.

At a livestock fair in 1906, 787 visitors guessed the dressed weight of an ox. Individual guesses varied widely, yet their mean, 1197 lb, was within one pound of the true weight, 1198 lb (Galton, *Vox Populi*, 1907). The same idea is used to label data: when three annotators look at a network event and two say *malicious* while one says *benign*, the majority label is kept, and strong disagreements are sent to an expert. Replace "annotators" by "fitted models" and you have an *ensemble*: several models produce several predictions, and a vote or an average produces one.

Two conditions make a crowd wise. The members must be *competent*: averaging random noise gives noise. And their errors must *differ*: if everybody makes the same mistake, the mistake survives the average. Three copies of the same model are not an ensemble; they have the same strengths and the same mistakes.

One fitted model can be unstable

Same algorithm, different sample, different boundary: that is variance.

Train the same learning algorithm on two different samples from the same population and the two fitted decision boundaries differ. An unstable learner fits the signal *and* the sample-specific noise; the noise part changes from sample to sample. In the language of Definition 3.3.1, the prediction at a fixed x has a large $\mathrm{Var}(x)$. Where positive and negative errors occur in different places for different fits, averaging the fits cancels part of the fluctuation and leaves the shared signal.

Key insight

Combining models helps when the component models are useful *and* their errors differ. Shared errors do not cancel. Every ensemble method is a recipe for producing competent models whose errors are as different as possible, plus a rule for combining them.

Three families

The methods of this chapter differ in *how* they obtain diversity and *how* they combine.

Family	How diversity is obtained	How predictions are combined
Voting / averaging (§5.2)	different algorithms, settings, features or initialisations, usually all fitted to the *same* data	fixed vote or average
Bagging / random forest (§5.4–5.5)	randomised *observations* (bootstrap samples) and randomised *candidate features*	fixed vote or average
Boosting (§5.7–5.9)	each new model targets the *current errors* of the ensemble	successive additive corrections

Two further architectures replace the fixed combination rule by a learned one: *stacking* (§5.12) learns one global combination of the base predictions, and a *mixture of experts* (§5.13) learns an input-dependent combination.

Summary: Why combine

- A single flexible fit has high variance: it reflects the sample it was trained on.
- Averaging competent models with *different* errors cancels part of the fluctuation; shared errors remain.
- Diversity comes from different models (voting), different data (bagging, random forests) or different targets (boosting); the combination is a vote, an average or a sum of corrections.

5.2 Averaging and Voting

What are we trying to get?

We want the two elementary combination rules, to see when hard and soft voting disagree and what each one throws away, and to *prove* how much averaging reduces variance as a function of the correlation between the component errors.

The rules

Definition 5.2.1 (Averaging; hard and soft voting)**.** Let $\widehat{f}_1, \dots, \widehat{f}_M$ be fitted models.

- *Regression (averaging):* $\widehat{f}_{\text{ens}}(x) = \dfrac{1}{M} \displaystyle\sum_{m=1}^{M} \widehat{f}_m(x)$.
- *Hard voting:* each classifier outputs a class $\hat{y}_m(x)$; the ensemble outputs the most frequent class.
- *Soft voting:* each classifier outputs posterior estimates; the ensemble averages them and takes the largest average,

$$\widehat{p}_{\text{ens}}(y = c \mid x) = \frac{1}{M} \sum_{m=1}^{M} \widehat{p}_m(y = c \mid x), \quad \hat{y}(x) = \arg\max_c \widehat{p}_{\text{ens}}(y = c \mid x).$$

Reading the formula

Hard voting counts heads; soft voting pools confidence. A classifier that says 0.49 and one that says 0.05 cast the *same* hard vote (class 0) but contribute very differently to a soft vote. Soft voting therefore uses strictly more information: the *margin* by which each model preferred its class. The price is that this information must be trustworthy: the probabilities should be reasonably calibrated (Chapter 3) and computed under the same class prior, otherwise one overconfident model dominates the average.

Example 5.2.2 (Hard and soft votes disagree)**.** Three classifiers evaluate the same flow with threshold 0.5:

model	$\widehat{p}(y=0 \mid x)$	$\widehat{p}(y=1 \mid x)$	prediction
A	0.51	0.49	0
B	0.52	0.48	0
C	0.05	0.95	1

Hard voting predicts class 0 (two votes to one). Soft voting averages $\widehat{p}(y=1 \mid x) = (0.49+0.48+0.95)/3 = 0.64$ and predicts class 1. The soft vote used the fact that A and B were coin flips while C was nearly certain; the hard vote discarded it. Whether that is right depends on whether C's 0.95 deserves to be believed.

Gotcha

Soft voting assumes comparable, reasonably calibrated probabilities under the same prior. A model that outputs 0.95 whenever it leans towards class 1 will dominate every average. Options: calibrate it (Chapter 3), weight the models, or fall back to hard voting, which caps each model at one vote and so is robust to overconfidence at the cost of discarding margins.

How much does averaging help? The variance formula

Averaging M independent errors divides the variance by M. Correlated errors leave a floor.

Fix an input x and let $e_m = \widehat{f}_m(x) - f(x)$ be the error of model m at x, viewed as a random variable over training samples. Suppose every error has the same variance and every pair the same correlation.

Theorem 5.2.3 (Variance of an average of correlated errors)**.** *Let $e_1, \dots, e_M$ have $\mathrm{Var}(e_m) = \sigma^2$ for all m and $\mathrm{Corr}(e_m, e_{m'}) = \rho$ for all $m \neq m'$. Then the error of the average, $\bar{e} = \frac{1}{M}\sum_m e_m$, satisfies*

$$\mathrm{Var}(\bar{e}) = \sigma^2\Big(\rho + \frac{1-\rho}{M}\Big).$$

In particular: $\rho = 0$ gives σ^2/M; $\rho = 1$ gives σ^2 (averaging changes nothing); and as $M \to \infty$ the variance tends to $\rho\sigma^2$, not to zero.

Proof. Expand the variance of a sum:

$$\mathrm{Var}\Big(\sum_{m=1}^{M} e_m\Big) = \sum_{m=1}^{M} \mathrm{Var}(e_m) + \sum_{m \neq m'} \mathrm{Cov}(e_m, e_{m'}) = M\sigma^2 + M(M-1)\rho\sigma^2,$$

because there are M variance terms and $M(M-1)$ ordered pairs, each with covariance $\rho\sigma^2$. Dividing by M^2,

$$\begin{aligned}\mathrm{Var}(\bar{e}) &= \frac{M\sigma^2 + M(M-1)\rho\sigma^2}{M^2} = \frac{\sigma^2}{M} + \frac{(M-1)\rho\sigma^2}{M} \\ &= \sigma^2\Big(\rho + \frac{1}{M} - \frac{\rho}{M}\Big) = \sigma^2\Big(\rho + \frac{1-\rho}{M}\Big).\end{aligned}$$

The three special cases follow by substitution and by letting $M \to \infty$. □

Key insight

Only the $M(M-1)$ cross terms carry the correlation. The M diagonal terms shrink like $1/M$ after division by M^2; the cross terms do not, because there are M^2 of them. The part of the error that all models *share* is immune to averaging.

Reading the formula

Read ρ as "how much of the error direction is shared". With $\rho = 0.2$ and $M = 25$ trees the variance is $\sigma^2(0.2 + 0.8/25) = 0.232\sigma^2$, not $\sigma^2/25 = 0.04\sigma^2$. Adding more trees moves $0.032\sigma^2$ towards zero but never touches the floor $0.2\sigma^2$. A useful ensemble therefore needs *accurate* components (σ^2 small) whose errors are *sufficiently different* (ρ small). Every later idea in this chapter, bootstrap samples, random feature subsets, sequential corrections, is a way of lowering ρ without raising σ^2 too much.

Interpretation & discussion

Theorem 5.2.3 is about variance only. Averaging does not remove bias: if every model is missing the same important feature, every error has the same sign and ρ is close to 1. This is the formal content of "shared errors do not cancel".

Where does diversity come from when all models see the same data?

Voting and averaging usually fit every model to the *same* training set. Diversity can still come from different learning algorithms (logistic regression, a tree, a nearest-neighbour rule), different settings or random initialisations, or different feature subsets. But because every model sees the same observations, they can all learn the same sample-specific pattern and make correlated errors. The next sections manufacture diversity more deliberately, by changing the *data* each model sees.

Summary: Averaging and voting

- Regression: average predictions. Classification: hard voting (count classes) or soft voting (average posteriors).
- Soft voting uses margins and needs calibrated, comparable probabilities; hard voting is robust to overconfidence but discards information.
- Var $= \sigma^2(\rho + (1-\rho)/M)$: independent errors give σ^2/M; correlation leaves a floor $\rho\sigma^2$ that no number of models removes.

5.3 Decision Trees as Base Learners

What are we trying to get?

Trees are the base learner of almost every ensemble in practice. We want to define a tree precisely as a partition with one constant per region, prove which constant is optimal in a leaf, state the greedy splitting rule with the Gini impurity, work one split by hand, and understand why trees are at once well suited to ensembles (flexible, fast, scale-free) and in need of them (high variance).

A tree is a partition with one number per region

Definition 5.3.1 (Decision tree)**.** A tree with J leaves partitions the feature space into disjoint regions $S_1, \dots, S_J$ and assigns one prediction θ_j to each region:

$$T(x) = \sum_{j=1}^{J} \theta_j \, \mathbf{1}[x \in S_j].$$

Each region is reached by a sequence of binary tests of the form $x_j \le t$ (go left) or $x_j > t$ (go right). The *depth* of a leaf is the number of tests on its path; the *maximum depth* and the *minimum leaf size* (fewest training observations allowed in a leaf) are the usual size controls.

Reading the formula

Exactly one indicator is 1 for any x, so $T(x)$ is simply "the constant of the region x falls in". A tree is a piecewise-constant function whose pieces are axis-aligned boxes. For classification θ_j is a class or a vector of class proportions; for regression it is a number.

Ideally one would choose the regions and their constants *jointly*,

$$\min_{J, \{S_j, \theta_j\}} \sum_{i=1}^{N} \ell\Big(y^{(i)}, \sum_{j=1}^{J} \theta_j \mathbf{1}[x^{(i)} \in S_j]\Big) + \lambda J,$$

with λJ a penalty on the number of leaves. The search over all partitions is infeasible; two simplifications make trees practical. First, for *fixed* regions the best constant in each leaf has a closed form. Second, the regions are chosen *greedily*, one split at a time.

Proposition 5.3.2 (Optimal leaf constant)**.** *For fixed regions the objective separates by leaf, so*

$$\theta_j^* = \arg\min_{\theta} \sum_{i:\, x^{(i)} \in S_j} \ell(y^{(i)}, \theta).$$

For squared-error loss θ_j^* is the mean *of the $y^{(i)}$ in leaf j; for absolute-error loss it is their* median*; for zero–one loss it is the* majority class*.*

Proof. Each training point lies in exactly one leaf, so the sum over i splits into J sums, the j-th of which involves only θ_j; minimise each separately. For squared error, $\frac{d}{d\theta} \sum_i (y^{(i)} - \theta)^2 = -2 \sum_i (y^{(i)} - \theta) = 0$ gives $\theta =$ mean (the finite-sample

version of Theorem 2.2.4). For absolute error the derivative is $\sum_i \operatorname{sign}(\theta - y^{(i)})$, which changes sign at the median. For zero–one loss the leaf should predict the class that is wrong on the fewest points, the majority. □

Greedy recursive splitting and the Gini impurity

Definition 5.3.3 (Impurity; Gini impurity; greedy split)**.** An *impurity* $I(S)$ measures how mixed the observations in a node S are. For classification with class proportions $\widehat{p}_c(S)$ the *Gini impurity* is

$$I(S) = 1 - \sum_{c=1}^{C} \widehat{p}_c(S)^2.$$

For regression the within-node squared error $\sum_{i \in S}(y^{(i)} - \bar{y}_S)^2$ plays the same role. At a node S, a candidate split is a feature j and threshold t giving $S_L = \{i \in S : x_j^{(i)} \le t\}$ and $S_R = S \setminus S_L$. The *greedy rule* chooses the split with the largest impurity reduction,

$$(j^*, t^*) = \arg\max_{j,t} \Big[I(S) - \frac{|S_L|}{|S|} I(S_L) - \frac{|S_R|}{|S|} I(S_R) \Big],$$

then repeats inside each child until a stopping rule (depth, leaf size, no useful split) is reached.

Reading the formula

$\sum_c \widehat{p}_c^2$ is the probability that two observations drawn from the node have the same class; 1 minus it is the probability they differ. A pure node ($\widehat{p}_c = 1$ for one class) has $I = 0$; two balanced classes give $I = 1 - \frac{1}{4} - \frac{1}{4} = 0.5$, the maximum for $C = 2$. The reduction formula compares the parent's impurity with the *size-weighted* average of the children's: a child that is pure but tiny counts little. Greedy means "best immediate reduction"; it does not look ahead and need not find the globally best tree.

Example 5.3.4 (Choosing a split by hand)**.** Eight observations, sorted by the candidate feature x:

x	1	2	3	4	5	6	7	8
y	0	0	0	1	0	1	1	1

The parent has four of each class, $I(S) = 1 - (\frac{1}{2})^2 - (\frac{1}{2})^2 = 0.50$.

- $x \le 3.5$: left $\{0,0,0\}$, $I = 0$; right $\{1,0,1,1,1\}$, $I = 1 - 0.8^2 - 0.2^2 = 0.32$. Weighted: $\frac{3}{8} \cdot 0 + \frac{5}{8} \cdot 0.32 = 0.20$. Reduction $0.50 - 0.20 = 0.30$.
- $x \le 4.5$: left $\{0,0,0,1\}$, $I = 1 - 0.75^2 - 0.25^2 = 0.375$; right $\{0,1,1,1\}$, $I = 0.375$. Weighted 0.375. Reduction 0.125.

The greedy rule prefers $x \le 3.5$: it gives a pure left leaf and the larger immediate reduction, even though $x \le 4.5$ splits the data in half.

Why trees in ensembles

Flexible and cheap, but one early split decides everything below it.

Trees capture nonlinear effects and feature interactions (a test on x_1 followed by a test on x_2 *is* an interaction), need no scaling of numerical features (only the order of values matters), fit complex patterns when grown deep, and are fast enough to fit hundreds of times. Their weakness is the one that averaging cures: a small change in the data can alter an early split, and everything below that split is rebuilt. Deep trees are therefore high-variance learners. Bootstrap samples make different trees; averaging makes the prediction stable while preserving the trees' ability to model nonlinearity and interactions.

Summary: Trees

- A tree is a partition into boxes with one constant per box; the optimal constant is the leaf mean (squared error), median (absolute error) or majority class.
- Splits are chosen greedily by impurity reduction; Gini $I = 1 - \sum_c \widehat{p}_c^2$ for classification, squared error for regression.
- Trees are flexible, scale-free and fast, but deep trees have high variance, which is exactly what ensembles reduce.

5.4 Bagging and Out-of-Bag Evaluation

What are we trying to get?

We want a way to give the *same* learner *different* training sets without throwing data away. Bootstrap sampling does this; we prove that each bootstrap sample omits about 36.8% of the observations, and we turn those omitted observations into a free, honest validation set: the out-of-bag estimate.

Bootstrap samples

Splitting $\mathcal{D}$ into M disjoint parts would give each model only N/M observations, too few unless N is huge. Instead each model is trained on a *perturbed copy* of the whole dataset.

Definition 5.4.1 (Bootstrap sample; bagging)**.** A *bootstrap sample* $\mathcal{D}^*$ of a training set $\mathcal{D}$ with N observations consists of N draws from $\mathcal{D}$ *with replacement*. Let $\text{count}(i)$ be the number of times observation i appears in $\mathcal{D}^*$; observations with $\text{count}(i) = 0$ are *out of bag* for that sample. *Bagging* (bootstrap aggregating) draws M bootstrap samples $\mathcal{D}_1^*, \dots, \mathcal{D}_M^*$, fits the same base learner to each, and aggregates:

$$\widehat{f}_{\text{bag}}(x) = \frac{1}{M}\sum_{m=1}^{M} \widehat{f}_m(x), \qquad \widehat{p}_{\text{bag}}(y = c \mid x) = \frac{1}{M}\sum_{m=1}^{M} \widehat{p}_m(y = c \mid x).$$

Example 5.4.2 (One bootstrap sample)**.** From $\mathcal{D} = \{1, \dots, 8\}$, eight draws with replacement might give $\mathcal{D}_1^* = \{2, 5, 2, 8, 1, 5, 7, 2\}$. Then

observation i	1	2	3	4	5	6	7	8
$\text{count}(i)$	1	3	0	0	2	0	1	1

Observation 2 is seen three times and 5 twice; 3, 4 and 6 are out of bag.

Different count patterns give different empirical distributions and so different fitted models.

Theorem 5.4.3 (Out-of-bag fraction)**.** *The probability that a given observation is absent from a bootstrap sample of size N is*

$$\mathbb{P}(\text{count}(i) = 0) = \left(1 - \frac{1}{N}\right)^N \xrightarrow[N\to\infty]{} e^{-1} \approx 0.368.$$

Hence a large bootstrap sample contains about $1 - e^{-1} \approx 63.2\%$ of the distinct *training observations, and about 36.8% are out of bag for each fitted model.*

Proof. Each draw picks observation i with probability $1/N$, so misses it with probability $1 - 1/N$. The N draws are independent, so the probability of missing i every time is $(1 - 1/N)^N$. The limit is the standard $\lim_{N\to\infty}(1 + x/N)^N = e^x$ with $x = -1$. Already for $N = 8$ the value is $(7/8)^8 = 0.344$; for $N = 100$ it is 0.366. □

Reading the formula

Two thirds in, one third out, for *every* model, and a *different* third each time. The "in" part is what makes the models different; the "out" part is what the next definition exploits.

Out-of-bag prediction and error

Definition 5.4.4 (Out-of-bag prediction)**.** For training observation i, let $\mathcal{M}_i = \{m : i \notin \mathcal{D}_m^*\}$ be the set of models whose bootstrap sample omits i; by Theorem 5.4.3, $|\mathcal{M}_i| \approx 0.368M$. The *out-of-bag (OOB) prediction* uses only those models,

$$\widehat{f}_{\text{OOB}}(x^{(i)}) = \frac{1}{|\mathcal{M}_i|} \sum_{m \in \mathcal{M}_i} \widehat{f}_m(x^{(i)}),$$

or, for classification, the average of their probabilities (or their vote). The *OOB error* is the loss of the OOB predictions over all training observations.

Reading the formula

For observation i, the OOB models never saw i, so their prediction at $x^{(i)}$ is a genuine unseen-data prediction, even though i is in the training set. Doing this for every i gives a validation estimate *without* holding anything out: the bootstrap already held out a third of the data for each model. It is a slight underestimate of the full ensemble's quality, since only about a third of the trees vote for each point.

Interpretation & discussion

The OOB error estimates the generalisation error of one fixed ensemble and can compare *ensemble-wide* choices: base-learner complexity, number of models, feature set. Two cautions. Repeated OOB-based choices overfit the OOB estimate just as repeated validation overfits a validation set (Chapter 3, Def-

inition 3.5.5): the final performance still needs an untouched test set. And the OOB estimate is tied to the training distribution; it says nothing about deployment shift. Bootstrap sampling provides internal validation cases, not a final test set.

One tree-size setting for the whole ensemble

Technique: Tune the ensemble, not the trees

Treat the tree-size setting ξ (maximum depth, minimum leaf size) as an *ensemble* hyperparameter: $\mathcal{D}_m^* \to \widehat{T}_m(x;\xi)$ with the *same* ξ for every m. For each candidate ξ, fit a complete bagged ensemble and compute its OOB error (or a validation error); pick the best ξ. Bagging usually uses deep trees, since averaging removes their variance; with noisy or limited data the OOB error may favour smaller trees or larger leaves. Tuning each tree separately would optimise the wrong target (a single tree's accuracy rather than the average's) and would push every tree towards the same strong, correlated fit.

Summary: Bagging

- Bootstrap: N draws with replacement; $(1 - 1/N)^N \to e^{-1}$, so about 63.2% distinct observations in, 36.8% out, for each model.
- Bagging fits the same learner to M bootstrap samples and averages or votes.
- OOB prediction for i uses only the $\approx 0.368M$ models that never saw i; the OOB error is a free internal validation estimate, but not a substitute for a test set.
- Tree size is one ensemble-level hyperparameter chosen by OOB or validation error.

5.5 Random Forests

What are we trying to get?

Bagged trees can still be too alike: a very strong feature ends up at the root of most of them and their errors stay correlated. We want the one extra randomisation that fixes this, its main hyperparameter m_{try}, and the resulting trade-off between tree strength and tree correlation in the light of Theorem 5.2.3.

Bagged trees can remain too similar

If one feature is very strong, almost every bootstrap sample will choose it near the root, and the trees that result have similar shapes and make similar mistakes. In Theorem 5.2.3 that is a large ρ: more trees stop helping early. The remedy is to *randomise the candidate features* at each split so that different trees are forced to start differently.

If x_1 is the best split everywhere, every bootstrap tree starts with x_1.

Definition 5.5.1 (Random forest)**.** For each tree $m = 1, \dots, M$:

1. draw a bootstrap sample $\mathcal{D}_m^*$;
2. at *each node*, select m_{try} of the d features uniformly at random;
3. choose the best split (Definition 5.3.3) using only those candidates;
4. grow the tree subject to leaf-size or depth constraints.

Aggregate the M trees by averaging probabilities or voting. A common starting value for classification is $m_{\text{try}} \approx \sqrt{d}$ (regression implementations often use $d/3$); with $d = 16$ each node receives four random candidates. A *new* subset is drawn at every node.

Reading the formula

Step 2 is the only difference from bagging. Setting $m_{\text{try}} = d$ recovers bagged trees. Small m_{try} means that at many nodes the strongest feature is simply not on offer, so an alternative is used, the trees take different shapes, and their errors decorrelate. The strong feature is not lost: it is available at a fraction m_{try}/d of the nodes and will be used deeper down.

Example 5.5.2 (Probability that a feature is offered)**.** With $d = 25$ and $m_{\text{try}} = 5$, a given feature is among the candidates at a node with probability $5/25 = 0.2$ (choosing 5 of 25 without replacement, the chance that a particular one is included is $\binom{24}{4}/\binom{25}{5} = 5/25$). It is *absent* from the root of one tree with probability 0.8, and from the roots of all $M = 10$ trees with probability $0.8^{10} \approx 0.107$. So in a ten-tree forest about 2 roots use it, while a bagged forest might use it at all 10.

Strength versus correlation

Large m_{try}	Small m_{try}
strongest features reused at most nodes	alternative features get considered
individual trees strong (small σ^2)	individual trees weaker (larger σ^2)
trees similar, errors correlated (large ρ)	trees less correlated (small ρ)
	too few candidates can give a poor split

Interpretation & discussion

Theorem 5.2.3 says the forest variance is $\sigma^2(\rho + (1-\rho)/M)$. Feature randomisation raises σ^2 a little and lowers ρ a lot; for moderate M the product improves. This is why a forest of individually *weaker* trees can beat a forest of individually *stronger* ones: the target is the average's error, not any tree's.

Controls

Hyperparameter	Main effect
number of trees M	stability and computation; OOB error decreases then plateaus; more trees stabilise rather than deepen the model
m_{try}	tree strength versus correlation; start near $\sqrt{d}$ for classification
minimum leaf size	prevents very small terminal regions
maximum depth	limits the number of successive splits

Technique: Forest-level selection

Tune m_{try}, minimum leaf size and, if useful, maximum depth by OOB performance or a separate validation set; each candidate setting requires fitting a complete forest. Increase the number of trees until the OOB error stabilises; beyond that, additional trees cost computation without changing the model's capacity.

Summary: Random forests

- Bagging plus a random subset of m_{try} candidate features at every node; $m_{\text{try}} = d$ is bagging.
- Feature randomisation weakens each tree slightly but decorrelates the trees; the forest improves because ρ falls faster than σ^2 rises.
- Start at $m_{\text{try}} \approx \sqrt{d}$; tune at forest level by OOB error; add trees until the OOB error plateaus.

5.6 Feature Importance

What are we trying to get?

A forest of hundreds of trees is not readable. We want two numerical summaries of *which features the fitted forest relies on*: one computed from the training process (mean decrease in impurity) and one computed by breaking a feature and watching the error (permutation importance), with the biases of each and the one interpretation both forbid.

Mean decrease in impurity

Definition 5.6.1 (Split credit; MDI)**.** For a node t containing N_t of the N training observations, split into children of sizes N_L, N_R, the *credit* of the split is its impurity reduction weighted by the fraction of data it acts on,

$$\Delta_t = \frac{N_t}{N}\Big[I(S_t) - \frac{N_L}{N_t} I(S_L) - \frac{N_R}{N_t} I(S_R)\Big].$$

The credit goes to the feature used at t. The *mean decrease in impurity* of feature j adds all credits of splits on x_j and averages across trees,

$$\text{MDI}_j = \frac{1}{M} \sum_{m=1}^{M} \sum_{t \in T_m:\ \text{split}(t)=j} \Delta_t,$$

usually normalised so that $\sum_j \text{MDI}_j = 1$.

Reading the formula

The bracket is the impurity reduction of Definition 5.3.3. The factor N_t/N says: a reduction achieved on all the data is worth more than the same reduction on 5% of it. The root has $N_t/N = 1$; deeper nodes have smaller weights but can still earn more credit through a larger reduction. The normalised

MDI is a share: "this feature produced 75% of all the impurity the forest removed".

Example 5.6.2 (Credit earned by one split)**.** A root node holds 10 flows, 5 attacks and 5 benign, Gini 0.50. The test "destination port = 22?" sends 4 flows left (0 attacks, 4 benign, Gini 0) and 6 right (5 attacks, 1 benign, Gini $1-(5/6)^2-(1/6)^2 = 10/36 = 0.278$). Weighted child impurity: $\frac{4}{10}(0) + \frac{6}{10}(0.278) = 0.167$. Since $N_t/N = 1$, $\Delta_t = 1 \cdot (0.50 - 0.167) = 0.333$, credited to destination port.

Example 5.6.3 (From credits to forest-level MDI)**.** Two trees produce the following credits. Tree 1: destination port 0.18, flow duration 0.06. Tree 2: destination port 0.12, total packets 0.04. Averaging over the two trees (a feature absent from a tree contributes 0 there): port $(0.18 + 0.12)/2 = 0.15$, duration $0.06/2 = 0.03$, packets $0.04/2 = 0.02$. The means sum to 0.20; dividing gives MDI = 0.75, 0.15, 0.10. Destination port produced the largest total impurity reduction.

Gotcha

MDI is free (it is computed during training) but biased: a feature with many possible split points (a continuous feature, a high-cardinality categorical) has more chances to find *some* reduction, by signal or by noise, and tends to be over-credited. MDI also measures the *training* fit, not generalisation.

Permutation importance

Definition 5.6.4 (Permutation importance)**.** For a fitted forest and a feature j:

1. compute the original OOB (or validation) error $\widehat{\mathrm{Err}}_{\mathrm{original}}$;
2. shuffle the values of x_j among the OOB observations, keeping every other column and the labels fixed;
3. predict again and measure $\widehat{\mathrm{Err}}_{\mathrm{shuffled}(j)}$.

The importance is the increase, $I_j = \widehat{\mathrm{Err}}_{\mathrm{shuffled}(j)} - \widehat{\mathrm{Err}}_{\mathrm{original}}$. No tree is retrained. In a forest each tree has its own OOB sample and its own shuffled copy; the increases are averaged across trees.

Reading the formula

Shuffling destroys the *information* in x_j (its link with the label and the other features) while preserving its marginal distribution, so the trees still receive plausible values and are not thrown off by out-of-range inputs. If the error barely moves, the forest was not relying on x_j; if it jumps, it was. Because this uses OOB or validation data and the fitted model as is, it measures reliance in *prediction*, not in training.

Example 5.6.5 (Shuffling destination port)**.** Four OOB flows with ports $80, 22, 443, 53$ become, after one shuffle, $53, 443, 80, 22$; duration, packets and

labels do not move. On 100 OOB flows the forest originally makes 10 errors (rate 0.10). Shuffling destination port gives 18 errors (0.18, increase 0.08); flow duration 13 (0.13, increase 0.03); total packets 11 (0.11, increase 0.01). Destination port matters most *for this fitted forest*: breaking it adds eight errors.

Gotcha

Two readings that importance does *not* support.

- **Correlated features mask one another.** If x_a and x_b carry the same information, shuffling x_a alone hurts little because the trees that use x_b (and the forest as a whole) still have the information. Both features then look less important than the information they share. Permute them jointly, or drop a column and refit, to see the shared contribution.
- **Importance is not causality.** A high score means the fitted model relies on the feature. The feature may be a proxy for the real cause (destination port stands in for the service being attacked), a consequence of the outcome recorded after the fact (leakage, Chapter 3), or an artefact of how the data were collected. Intervening on it would not change the outcome.

Summary: Feature importance

- MDI: sum of $\frac{N_t}{N} \times$(impurity reduction) over a feature's splits, averaged over trees, normalised; fast, training-based, biased towards features with many split points.
- Permutation importance: error increase on OOB/validation data after shuffling one feature; no retraining; measures reliance of the fitted model.
- Correlated features mask each other; importance does not imply causality.

5.7 Boosting for Regression

What are we trying to get?

Bagging fits many models *independently* and averages. Boosting asks a different question: *what should the next model learn from the current mistakes?* We want the additive model this leads to, a proof that for squared error "fit the next tree to the residuals" is exactly minimising the loss, the gradient-boosting algorithm, one iteration by hand, and the three hyperparameters that jointly control overfitting.

A chain of specialists

Tree 1 gets the trend. Tree 2 fixes what tree 1 got wrong. Tree 3 fixes what is still wrong.

The observations stay fixed. A first learner makes a baseline prediction, right in general but wrong in places. Compute the *leftovers*, the residuals $r^{(i)} = y^{(i)} - \hat{y}^{(i)}$. Train a second learner exclusively to predict those residuals; its job is to correct the first. A third learner is trained on what remains after the first two, and so on. Each added learner changes the ensemble's prediction where the current

errors are, and nowhere else. Bagging builds models in parallel; boosting adds corrections sequentially.

Definition 5.7.1 (Additive boosted model)**.** A boosted ensemble after m rounds is

$$F_m(x) = F_{m-1}(x) + \eta\, T_m(x),$$

where T_m is the new tree fitted in round m and $\eta \in (0, 1]$ is the *learning rate* (shrinkage). Unrolled, $F_M(x) = F_0(x) + \eta \sum_{m=1}^{M} T_m(x)$: a constant plus a scaled *sum* of trees.

Reading the formula

Compare with bagging's $\frac{1}{M}\sum_m \widehat{f}_m$: there, every model predicts the *whole* target and the average smooths them; here, every tree predicts a *correction* and the corrections are summed. Averaging reduces variance; summing corrections reduces the remaining loss (mostly bias) one step at a time. The learning rate η scales each step down; $\eta = 1$ applies the full correction, $\eta = 0.1$ one tenth of it.

Theorem 5.7.2 (Squared error: boosting fits residuals)**.** *With squared-error loss* $L = \frac{1}{2}\sum_{i=1}^{N} \left(y^{(i)} - F_m(x^{(i)})\right)^2$ *and the update* $F_m = F_{m-1} + \eta T_m$*,*

$$L = \frac{1}{2}\sum_{i=1}^{N} \Big(\underbrace{y^{(i)} - F_{m-1}(x^{(i)})}_{r_m^{(i)}} - \eta T_m(x^{(i)}) \Big)^2,$$

so minimising L *over* T_m *is the same as fitting* ηT_m *to the current residuals* $r_m^{(i)}$ *by least squares. In practice* T_m *is fitted to* $r_m^{(i)}$ *and* η *is applied afterwards in the update.*

Proof. Substitute $F_m(x^{(i)}) = F_{m-1}(x^{(i)}) + \eta T_m(x^{(i)})$ into L and group $y^{(i)} - F_{m-1}(x^{(i)})$, which is a fixed number at round m because F_{m-1} is already fitted. The expression is then the least-squares objective of a regression whose *targets* are the residuals and whose *predictor* is ηT_m. □

Key insight

At round m everything built so far is frozen. The only moving part is T_m, and the loss depends on it only through "residual minus correction". The next tree therefore sees a plain regression problem whose targets are the current errors.

The gradient boosting algorithm for regression

Definition 5.7.3 (Gradient boosting for regression)**.**

1. Initialise with the best constant, $F_0(x) = \arg\min_w \sum_i \ell(y^{(i)}, w)$; for squared error this is the mean $\bar{y}$ (Proposition 5.3.2).
2. For $m = 1, \ldots, M$:
   (a) compute the pseudo-residuals $r_m^{(i)} = -\partial \ell(y^{(i)}, F)/\partial F\big|_{F=F_{m-1}(x^{(i)})}$; for squared error this is the raw error $r_m^{(i)} = y^{(i)} - F_{m-1}(x^{(i)})$;

(b) fit a regression tree T_m to the targets $r_m^{(i)}$, creating leaf regions $S_{1,m}, \dots, S_{Q_m,m}$;

(c) in each leaf, set the output to the mean residual, $w_{q,m} = \frac{1}{|S_{q,m}|} \sum_{i \in S_{q,m}} r_m^{(i)}$;

(d) update $F_m(x) = F_{m-1}(x) + \eta T_m(x)$, where $T_m(x) = \sum_q w_{q,m} \mathbf{1}[x \in S_{q,m}]$.

Reading the formula

The word *gradient*: $-\partial \ell / \partial F$ is the direction in which changing the prediction at $x^{(i)}$ decreases the loss fastest. For squared error $\ell = \frac{1}{2}(y - F)^2$ the negative derivative is $y - F$, the residual, which is why "fit the residuals" and "fit the negative gradient" coincide here. For other losses they differ, and the general recipe is: fit the tree to the negative gradient, then choose each leaf's output to minimise the actual loss (§5.8).

Example 5.7.4 (One boosting round by hand)**.** Three observations with learning rate $\eta = 0.1$:

ID	x	y	F_0	$r_1 = y - F_0$
1	1.2	10	20	-10
2	4.5	20	20	0
3	7.1	30	20	$+10$

Initialise: $F_0 = \text{mean}(10, 20, 30) = 20$. *Residuals:* as tabulated. *Fit* T_1*:* a split at $x < 2.85$ puts ID 1 in leaf S_1 and IDs 2, 3 in leaf S_2. *Leaf weights:* $w_{1,1} = -10$, $w_{2,1} = (0 + 10)/2 = +5$. *Update:* for ID 3, $F_1(7.1) = 20 + 0.1 \cdot 5 = 20.5$, closer to 30; for ID 1, $F_1(1.2) = 20 - 0.1 \cdot 10 = 19$. The new residuals are $-9, -0.5, 9.5$; the next tree is fitted to those.

Why many shallow trees rather than one deep tree

One deep tree	Boosted shallow trees
fits the whole rule at once	tree 1 isolates the loudest pattern; later trees target leftovers
hierarchical lock-in: early splits force rigid boundaries, deep splits only refine the sub-region they sit in	*global adaptability*: each new tree may adjust the boundary anywhere in feature space
a poor early split cannot be revised and easily amplifies noise	shrinkage η enforces gradual convergence

Instead of committing to an architecture in one step, boosting builds a flexible decision surface incrementally. This is the bias–variance trade-off of Chapter 3 in a new guise: each shallow tree has high bias and low variance, and the sum of many of them lowers the bias while η and early stopping keep the variance in check.

Overfitting and the interconnected triad

Boosting can continue indefinitely and the training loss keeps decreasing; after some number of rounds the validation loss starts rising because new trees fit noise.

Training loss keeps falling forever; validation loss turns up. Stop there.

Early stopping (Definition 3.4.5) selects the number of rounds at the validation minimum. Three hyperparameters control capacity *jointly*:

- **Number of rounds M.** Too small underfits; too large overfits.
- **Learning rate η.** Smaller steps need more rounds to reach the same fit: roughly $\eta \propto 1/M$. Smaller η usually generalises slightly better at a higher computational cost.
- **Tree depth or maximum leaves Q.** Limits the order of feature interactions a tree can express (depth 1: additive effects only; depth 2: pairwise interactions). Shallow trees, depth 2 to 6, usually generalise best.

Technique: Do not tune in isolation

The three are coupled: halving η shifts the optimal M up; deeper trees reach the validation minimum in fewer rounds. Use a grid or randomised search over (η, Q), with M chosen by early stopping on validation or by cross-validation (Definition 3.5.5) for each candidate, and compare configurations by their validation loss at the selected round.

Summary: Boosting for regression

- $F_m = F_{m-1} + \eta T_m$: a sum of corrections, not an average of predictions.
- For squared error, minimising the loss over T_m is least-squares fitting of the residuals; the pseudo-residual is the negative gradient of the loss.
- Initialise at the mean; each leaf outputs the mean residual; scale by η.
- Many shallow trees avoid the lock-in of one deep tree; M, η, depth are tuned together with early stopping.

5.8 Boosting for Binary Classification

What are we trying to get?

Probabilities live in $(0, 1)$ and cannot be summed with tree outputs. We want the fix (boost the *logit*), the gradient and Hessian of the cross-entropy loss with respect to the logit (proved), why the leaf output no longer has a closed form, and the second-order (Newton) shortcut that restores one, first without and then with regularisation.

Boost in logit space

Definition 5.8.1 (Boosting the logit; binary cross-entropy)**.** The ensemble produces an unconstrained score $F^{(i)} \in (-\infty, \infty)$ for observation i and maps it to a probability through the sigmoid (Definition 4.2.1),

$$\widehat{p}^{(i)} = \sigma(F^{(i)}) = \frac{1}{1 + e^{-F^{(i)}}}.$$

The loss is the binary cross-entropy (Definition 2.5.8) written as a function of the logit,

$$\ell^{(i)}(y^{(i)}, F^{(i)}) = -\Big[y^{(i)} \ln \widehat{p}^{(i)} + (1 - y^{(i)}) \ln(1 - \widehat{p}^{(i)})\Big].$$

Trees are added to F: $F_m = F_{m-1} + \eta T_m$, and $\widehat{p}$ is recomputed from F_m.

Reading the formula

Adding a tree output to a probability could give 1.3. Adding it to a logit is always legal, and the sigmoid brings the result back inside $(0, 1)$. This is exactly the discriminative route of Chapter 4, with the score $s_\theta(x)$ now a sum of trees instead of $w^\top x + b$.

Theorem 5.8.2 (Gradient and Hessian of the cross-entropy in the logit)**.** *With $\widehat{p} = \sigma(F)$,*

$$g = \frac{\partial \ell}{\partial F} = \widehat{p} - y, \qquad h = \frac{\partial^2 \ell}{\partial F^2} = \widehat{p}(1 - \widehat{p}).$$

Proof. First, $\sigma'(F) = \sigma(F)(1 - \sigma(F))$: writing $\sigma = (1 + e^{-F})^{-1}$, $\sigma' = e^{-F}(1 + e^{-F})^{-2} = \sigma \cdot \frac{e^{-F}}{1+e^{-F}} = \sigma(1 - \sigma)$. Then by the chain rule

$$\begin{aligned} \frac{\partial \ell}{\partial F} = \frac{\partial \ell}{\partial \widehat{p}} \frac{d\widehat{p}}{dF} &= \Big(-\frac{y}{\widehat{p}} + \frac{1-y}{1-\widehat{p}} \Big) \widehat{p}(1 - \widehat{p}) \\ &= -y(1 - \widehat{p}) + (1 - y)\widehat{p} = -y + y\widehat{p} + \widehat{p} - y\widehat{p} = \widehat{p} - y. \end{aligned}$$

Differentiating once more, $\partial(\widehat{p} - y)/\partial F = d\widehat{p}/dF = \widehat{p}(1 - \widehat{p})$. □

Reading the formula

The *negative* gradient $-g = y - \widehat{p}$ is the classification analogue of the residual: how far the current probability is from the label. The Hessian $h = \widehat{p}(1 - \widehat{p})$ is the variance of a Bernoulli with parameter $\widehat{p}$: it peaks at 0.25 when $\widehat{p} = 0.5$ (the model is unsure, the loss curves most) and vanishes when $\widehat{p} \to 0$ or 1 (confident, loss flat in F). For $y = 1$ and a very negative F (confidently wrong), $-g \to 1$ is at its largest while $h \to 0$: a large residual with almost no curvature. Keep this picture; it explains both the Newton step and why it needs regularisation.

Basic gradient boosting and the line-search bottleneck

Definition 5.8.3 (Gradient boosting for classification)**.** At round m:

1. pseudo-residuals $r_m^{(i)} = -\partial \ell^{(i)}/\partial F_{m-1}^{(i)} = y^{(i)} - \widehat{p}_{m-1}^{(i)}$;
2. grow a regression tree on $r_m^{(i)}$ by squared-error splits, giving leaves $S_{1,m}, \dots, S_{Q_m,m}$ (*split search determines the regions*);
3. with the structure fixed, choose each leaf output by a *line search* on the true loss,
$$w_{q,m} = \arg\min_{w} \sum_{i \in S_{q,m}} \ell^{(i)}(y^{(i)}, F_{m-1}^{(i)} + w)$$
(*line search determines the outputs*);
4. $T_m(x) = \sum_q w_{q,m} \mathbf{1}[x \in S_{q,m}]$ and $F_m = F_{m-1} + \eta T_m$.

For squared error step 3 had the closed form "mean residual". For log loss it has none: $\sum_i \ell(y^{(i)}, F^{(i)} + w)$ is a sum of log-sigmoids in w and must be minimised

numerically, leaf by leaf, with repeated loss evaluations. This is the *line-search bottleneck*. The remedy is to replace the loss by a local quadratic.

The Newton step: a quadratic approximation gives a closed form

Theorem 5.8.4 (Newton leaf value)**.** *Expand the loss of observation i to second order around the current logit,*

$$\ell^{(i)}(y^{(i)}, F_{m-1}^{(i)} + w) \;\approx\; \ell^{(i)}(y^{(i)}, F_{m-1}^{(i)}) + g^{(i)} w + \tfrac{1}{2} h^{(i)} w^2.$$

Minimising the sum of these approximations over a leaf $S_{q,m}$ gives

$$w_{q,m} = \frac{\sum_{i \in S_{q,m}} (-g^{(i)})}{\sum_{i \in S_{q,m}} h^{(i)}} = \frac{\sum_{i \in S_{q,m}} (y^{(i)} - \widehat{p}_{m-1}^{(i)})}{\sum_{i \in S_{q,m}} \widehat{p}_{m-1}^{(i)} (1 - \widehat{p}_{m-1}^{(i)})}.$$

Adding an L_2 penalty $\frac{\lambda}{2} w^2$ to the leaf objective gives the regularised *value*

$$w_{q,m} = \frac{\sum_{i \in S_{q,m}} (-g^{(i)})}{\sum_{i \in S_{q,m}} h^{(i)} + \lambda}.$$

Proof. The sum over the leaf, $\sum_i \left[\ell^{(i)} + g^{(i)} w + \frac{1}{2} h^{(i)} w^2\right] + \frac{\lambda}{2} w^2$, is a quadratic in the single number w with positive leading coefficient $\frac{1}{2}(\sum_i h^{(i)} + \lambda)$, since $h^{(i)} > 0$. Its derivative $\sum_i g^{(i)} + (\sum_i h^{(i)} + \lambda) w$ vanishes at $w = -\sum_i g^{(i)} / (\sum_i h^{(i)} + \lambda)$. Setting $\lambda = 0$ gives the unregularised step; substituting Theorem 5.8.2 gives the explicit form. □

Reading the formula

Numerator: the total residual in the leaf, how far and in which direction the probabilities are off. Denominator: the total curvature, how fast the loss bends. A Newton step is "residual divided by curvature": in a leaf full of unsure observations ($h \approx 0.25$ each) the step is moderate; in a leaf of confidently-wrong observations ($h \approx 0$) the denominator collapses and the step *explodes*. That is the risk: small leaves with tiny $\sum h$ give enormous outputs and extreme overfitting. The λ in the denominator is a floor that caps the step; it matters most exactly where $\sum h$ is smallest.

Example 5.8.5 (Newton leaf value by hand)**.** A leaf holds $(y, \widehat{p}) = (1, 0.2), (1, 0.4), (0, 0.8)$. Gradients $g = \widehat{p} - y$: $-0.8, -0.6, +0.8$, so $G = \sum g = -0.6$. Hessians $h = \widehat{p}(1 - \widehat{p})$: $0.16, 0.24, 0.16$, so $H = 0.56$. Unregularised leaf value $-G/H = 0.6/0.56 = 1.071$; with $\lambda = 1$, $0.6/1.56 = 0.385$. With $\eta = 0.1$ the logit actually moves by 0.0385 (regularised). The positive sign raises all three probabilities, helping the two positives more than it hurts the negative.

Interpretation & discussion

Both GBM variants still build the *tree structure* by unweighted squared-error splits on the raw residuals $y - \widehat{p}$, and only afterwards compute Newton outputs and apply λ to the frozen leaves. The split search is blind to the curvature information that the leaf values use. XGBoost's contribution (§5.9) is to use one objective for both.

Summary: Boosting for classification

- Boost the logit F; $\widehat{p} = \sigma(F)$; loss is binary cross-entropy.
- $g = \widehat{p} - y$ (so the pseudo-residual is $y - \widehat{p}$); $h = \widehat{p}(1-\widehat{p})$, maximal at $\widehat{p} = 0.5$, zero when confident.
- Exact leaf outputs need a numerical line search; a second-order expansion gives the Newton step $w = \sum(-g)/\sum h$.
- Small $\sum h$ makes the step explode; the regularised step $\sum(-g)/(\sum h+\lambda)$ caps it.

5.9 XGBoost: One Objective for Structure and Leaf Values

What are we trying to get?

We want the single regularised second-order objective that XGBoost uses to evaluate a *whole tree*, to show (by completing the square) that it is a weighted least-squares problem with a new target and a new weight, to derive the optimal leaf value and the *gain* of a split, and to see precisely what the two penalties λ and γ do.

The objective

Definition 5.9.1 (XGBoost tree objective)**.** A proposed tree with leaves $S_{1,m}, \dots, S_{Q_m,m}$ and outputs $w_{q,m}$ is scored by

$$\mathcal{L}_m = \sum_{q=1}^{Q_m} \sum_{i:\, x^{(i)} \in S_{q,m}} \left[g^{(i)} w_{q,m} + \tfrac{1}{2} h^{(i)} w_{q,m}^2 \right] + \gamma Q_m + \frac{\lambda}{2} \sum_{q=1}^{Q_m} w_{q,m}^2,$$

with $g^{(i)}, h^{(i)}$ the first and second derivatives of the loss at the current prediction $F_{m-1}(x^{(i)})$.

Reading the formula

Three parts. The double sum is the second-order Taylor approximation of the total loss change produced by the tree (the constant $\sum_i \ell^{(i)}$ is dropped since it does not depend on the tree). γQ_m charges γ per leaf. $\frac{\lambda}{2}\sum_q w_q^2$ charges for large outputs. The *same* number scores the structure (through which observations share a leaf) and the outputs; XGBoost therefore uses gradients, Hessians and regularisation *while growing* the tree, not after.

Theorem 5.9.2 (The objective is a weighted regression)**.** *For observation i in leaf q,*

$$g^{(i)} w_q + \tfrac{1}{2} h^{(i)} w_q^2 = \tfrac{1}{2} h^{(i)} \left(r_{\mathrm{XGB}}^{(i)} - w_q \right)^2 + \mathit{const}, \quad r_{\mathrm{XGB}}^{(i)} = -\frac{g^{(i)}}{h^{(i)}} = \frac{y^{(i)} - \widehat{p}_{m-1}^{(i)}}{\widehat{p}_{m-1}^{(i)}(1 - \widehat{p}_{m-1}^{(i)})}.$$

Hence

$$\mathcal{L}_m = \sum_q \sum_{i \in S_{q,m}} \tfrac{1}{2} h^{(i)} \big(r_{\mathrm{XGB}}^{(i)} - w_{q,m}\big)^2 + \gamma Q_m + \frac{\lambda}{2} \sum_q w_{q,m}^2 + \textit{constant.}$$

Proof. Complete the square: $\frac{1}{2}h(w+g/h)^2 = \frac{1}{2}hw^2+gw+g^2/(2h)$, so $gw+\frac{1}{2}hw^2 = \frac{1}{2}h(w-(-g/h))^2 - g^2/(2h)$, and $-g^2/(2h)$ does not involve w. Substituting $g = \widehat{p} - y$ and $h = \widehat{p}(1-\widehat{p})$ gives the explicit target. □

Reading the formula

Each observation asks the tree to output $r_{\mathrm{XGB}} = -g/h$, the one-step Newton correction *for that observation alone*, and its request is weighted by h. An unsure observation (h large) has a modest target and a loud voice; a confidently-wrong one has a huge target ($(y-\widehat{p})/(\widehat{p}(1-\widehat{p})) \approx 1/\widehat{p}$ when $y=1$, $\widehat{p} \to 0$) but a whisper of a weight. Target and weight must always be read together: their product, $h \cdot r_{\mathrm{XGB}} = -g = y - \widehat{p}$, is bounded by 1, which is why the leaf value stays finite. The tree is grown by *weighted* squared-error splits on these targets, which is what "XGBoost uses the Hessian in the split search" means.

Leaf values and the gain of a split

Theorem 5.9.3 (Optimal leaf value and split gain)**.** *For a leaf q let $G_q = \sum_{i \in S_q} g^{(i)}$ and $H_q = \sum_{i \in S_q} h^{(i)}$. The leaf's contribution to $\mathcal{L}_m$ is $\phi_q(w) = G_q w + \frac{1}{2}(H_q+\lambda)w^2$, minimised at*

$$w_q^* = -\frac{G_q}{H_q + \lambda}, \qquad \phi_q(w_q^*) = -\frac{1}{2}\,\frac{G_q^2}{H_q+\lambda}.$$

Splitting a parent with (G,H) into children (G_L,H_L) and (G_R,H_R), where $G = G_L + G_R$ and $H = H_L + H_R$, changes the objective by $-$gain, with

$$\text{gain} = \frac{1}{2}\Big[\frac{G_L^2}{H_L+\lambda} + \frac{G_R^2}{H_R+\lambda} - \frac{G^2}{H+\lambda}\Big] - \gamma.$$

The split is accepted only when gain > 0. *After the tree is grown, $F_m = F_{m-1} + \eta T_m$.*

Proof. Summing the per-observation terms over a leaf gives $\sum_i g^{(i)} w + \frac{1}{2}\sum_i h^{(i)} w^2 + \frac{\lambda}{2}w^2 = G_q w + \frac{1}{2}(H_q+\lambda)w^2$. Its derivative $G_q + (H_q+\lambda)w$ vanishes at w_q^*; substituting, $\phi_q(w_q^*) = -G_q^2/(H_q+\lambda) + \frac{1}{2}G_q^2/(H_q+\lambda) = -\frac{1}{2}G_q^2/(H_q+\lambda)$. Replacing one leaf by two changes the data-fit part from $\phi_{\mathrm{parent}}(w^*)$ to $\phi_L(w_L^*) + \phi_R(w_R^*)$, a decrease of $\frac{1}{2}[G_L^2/(H_L+\lambda) + G_R^2/(H_R+\lambda) - G^2/(H+\lambda)]$, and raises the leaf count from Q_m to $Q_m + 1$, so the penalty rises by $\gamma(Q_m+1) - \gamma Q_m = \gamma$. □

Reading the formula

$-\frac{1}{2}G^2/(H+\lambda)$ is "how much loss this leaf can remove with its best output"; more negative is better. A split is worth taking when the two children can jointly remove more than the parent could, by more than the price γ of the extra leaf. The two penalties act differently: λ sits inside every denominator

and *shrinks* all leaf outputs and all three terms, most strongly where H is small; γ is a flat per-split threshold that *discourages* splits whose improvement is marginal. In one line: λ shrinks leaf outputs; γ discourages unnecessary splits.

Example 5.9.4 (Does the split pay for its leaf?)**.** Parent $(G, H) = (-4, 3)$, children $(G_L, H_L) = (-3, 1)$ and $(G_R, H_R) = (-1, 2)$, $\lambda = 1$, $\gamma = 0.2$. Parent: $w^* = 4/4 = 1$, $\phi = -\frac{1}{2} \cdot 16/4 = -2$. Left: $w^* = 3/2$, $\phi = -\frac{1}{2} \cdot 9/2 = -2.25$. Right: $w^* = 1/3$, $\phi = -\frac{1}{2} \cdot 1/3 = -0.167$. Children together -2.417; the data fit improves by 0.417; gain $= 0.417 - 0.2 = 0.217 > 0$, accept. The split would be rejected for any $\gamma \geq 5/12 \approx 0.417$.

The four leaf-value mechanisms side by side

Algorithm	Structure target (split search)	Leaf value mechanism
Basic boosting	raw error $y - \widehat{p}$	exact numerical line search
Unregularised GBM	raw error $y - \widehat{p}$	Newton step $\sum(-g)/\sum h$
Regularised GBM	raw error $y - \widehat{p}$	regularised step $\sum(-g)/(\sum h + \lambda)$
XGBoost	Newton target $r_{\text{XGB}} = -g/h$, weighted by h	same regularised step, found in the same weighted pass that chose the splits; gain includes γ

XGBoost unifies structure selection and weight optimisation into a single pass over one objective, which is both faster and better regularised.

Further controls and implementation

Beyond λ and γ, XGBoost offers *row subsampling* (a random fraction of observations per tree), *feature subsampling* (by tree, level or split, the random-forest idea inside boosting), *minimum child weight* (reject a split if either child's H is too small, which directly prevents the exploding step), and *tree-size limits* (maximum depth or leaves). Its speed comes from engineering: histogram-based split search, sparse and missing-value handling, parallel split evaluation across features, and GPU support. Boosting rounds themselves remain sequential.

Summary: XGBoost

- One objective: second-order loss approximation $+ \gamma Q + \frac{\lambda}{2} \sum w_q^2$, used for splits *and* outputs.
- Completing the square: a weighted least-squares problem with target $-g/h$ and weight h; their product $y - \widehat{p}$ is bounded.
- $w_q^* = -G_q/(H_q + \lambda)$; gain $= \frac{1}{2}[G_L^2/(H_L + \lambda) + G_R^2/(H_R + \lambda) - G^2/(H + \lambda)] - \gamma$; accept if positive.
- λ shrinks outputs (continuous), γ charges a fixed price per extra leaf (discrete); row/feature subsampling and minimum child weight add further control.

5.10 Multi-Class Boosting and Other Libraries

What are we trying to get?

We want to extend logit boosting from two classes to K: one tree per class per round, class-specific gradients and Hessians, and the diagonal approximation of the softmax Hessian that lets the K trees be fitted separately. Then a map of the other names you will meet.

The softmax space

For K classes the ensemble maintains a vector of logits $F_1, \dots, F_K$ and maps it to probabilities with the softmax (Definition 4.2.4), $\widehat{p}_k = e^{F_k} / \sum_j e^{F_j}$. The loss is the multi-class cross-entropy with one-hot label y_k, $\ell = -\sum_k y_k \ln \widehat{p}_k$. Instead of one tree per round, the algorithm builds K trees per round, one per logit.

Proposition 5.10.1 (Class-specific derivatives)**.** *For one observation,*

$$g_k = \frac{\partial \ell}{\partial F_k} = \widehat{p}_k - y_k, \qquad \frac{\partial^2 \ell}{\partial F_k \, \partial F_r} = \begin{cases} \widehat{p}_k(1 - \widehat{p}_k) & k = r, \\ -\widehat{p}_k \widehat{p}_r & k \neq r. \end{cases}$$

Using only the diagonal, $h_k = \widehat{p}_k(1 - \widehat{p}_k)$, the XGBoost target for class k is $r_{\mathrm{XGB},k} = -g_k/h_k = (y_k - \widehat{p}_k)/(\widehat{p}_k(1 - \widehat{p}_k))$, formally identical to the binary case.

Proof. Write $\ell = -\sum_k y_k F_k + \ln \sum_j e^{F_j}$ (since $\ln \widehat{p}_k = F_k - \ln \sum_j e^{F_j}$ and $\sum_k y_k = 1$). Then $\partial \ell / \partial F_k = -y_k + e^{F_k} / \sum_j e^{F_j} = \widehat{p}_k - y_k$. Differentiating $\widehat{p}_k = e^{F_k} / \sum_j e^{F_j}$ with respect to F_r by the quotient rule gives $\widehat{p}_k(\mathbf{1}[k = r] - \widehat{p}_r)$, which is $\widehat{p}_k(1 - \widehat{p}_k)$ on the diagonal and $-\widehat{p}_k \widehat{p}_r$ off it. □

Reading the formula

The softmax *couples* the classes: raising F_1 lowers every other $\widehat{p}_k$, which is what the off-diagonal terms $-\widehat{p}_k \widehat{p}_r$ record. Keeping the full Hessian would make the K leaf values of one round depend on each other. Dropping the off-diagonal terms, $H \approx \mathrm{diag}(\widehat{p}_1(1 - \widehat{p}_1), \dots, \widehat{p}_K(1 - \widehat{p}_K))$, makes the approximate objective a *sum* of K independent binary-style objectives, so one tree can be fitted per class. The coupling is restored at the end of the round when the softmax is recomputed jointly from all updated logits.

Technique: One multi-class round

(1) From the current scores $F_{m-1,1}, \dots, F_{m-1,K}$ compute all class gradients and Hessians. (2) Fit one class-specific tree for each of the K separated objectives; these K fits, and the split evaluation inside each, can run in parallel. (3) Update all K scores and recompute the softmax probabilities before round $m+1$. Rounds stay sequential because round $m+1$ needs the updated probabilities. Multi-class boosting does K times the work per round; parallelism reduces elapsed time, not total work.

Other boosting approaches and libraries

Name	Type	Main emphasis
AdaBoost	a boosting *algorithm*	increases the weights of misclassified observations so that the next weak classifier focuses on them; the ensemble is a weighted vote
LightGBM	gradient-boosted-tree *library*	histogram-based splits and leaf-wise growth for large datasets
CatBoost	gradient-boosted-tree *library*	ordered treatment of categorical features designed to limit target leakage

Gotcha

AdaBoost is a different boosting *algorithm* (reweight observations, combine classifiers by weighted vote). XGBoost, LightGBM and CatBoost are *libraries* implementing gradient-boosted trees with different engineering choices. Do not list them as four algorithms of the same kind.

Summary: Multi-class boosting

- K logits, softmax, cross-entropy; K trees per round.
- $g_k = \widehat{p}_k - y_k$; the softmax Hessian couples classes through $-\widehat{p}_k\widehat{p}_r$; its diagonal $\widehat{p}_k(1-\widehat{p}_k)$ separates the K objectives.
- AdaBoost is an algorithm; XGBoost, LightGBM, CatBoost are libraries.

5.11 Bagging versus Boosting, and When Ensembles Do Not Help

What are we trying to get?

We want the two families side by side, a decision procedure that starts from the *source* of the error rather than from a favourite method, and the honest list of situations in which no ensemble will rescue a model.

Question	Bagging / random forest	Boosting
training	parallel randomised fits	sequential corrections
main mechanism	average unstable predictions (variance)	reduce the current loss (mostly bias)
typical trees	deep or moderately deep	shallow
diversity source	bootstrap rows, feature subsets	changing residuals or gradients
main controls	tree strength versus correlation (m_{try}, leaf size)	learning rate, depth, rounds
noise sensitivity	generally robust	can focus heavily on difficult or noisy cases
parallelisation	natural across trees	limited by sequential dependence

These are tendencies: either method can affect both bias and variance.

Technique: Start from the source of error

1. Is one strong learner *unstable* across training samples? Try bagging or a random forest.
2. Can many small corrections improve a *structured residual pattern*? Try gradient-boosted trees.
3. Do credible but different model families make *complementary* errors? Try validated voting or averaging.
4. Is the dominant error caused by *missing information* or *deployment shift*? An ensemble alone is unlikely to solve it.

Gotcha

Ensembles do not help when

- all models are missing the same important information (every error shares a direction; $\rho \approx 1$);
- the models are too similar and make the same mistakes;
- boosting starts fitting noise, outliers or incorrect labels (it treats every residual as signal);
- validation data are reused so often that the choices overfit them;
- test or deployment data differ substantially from the training data.

Additional models also cost computation, memory and system complexity, which Lab 3 measures explicitly.

Summary: Choosing

- Bagging and forests average parallel, randomised, deep fits; boosting sums sequential, shallow corrections.
- Diagnose first: instability, structured residuals, complementary families, or missing information and shift.
- No ensemble repairs shared missing information or a distribution shift.

5.12 Stacking

What are we trying to get?

Voting fixes the combination rule in advance. We want to *learn* it: a meta-model that takes the base predictions as inputs, and the one procedural rule that keeps this honest, out-of-fold predictions.

Definition 5.12.1 (Stacking)**.** Base models $m = 1, \ldots, M$ (e.g. logistic regression, a random forest, gradient boosting) produce predictions z_m, usually numbers or class probabilities. A *meta-model* h_γ maps them to the final prediction, $\hat{y} = h_{\hat{\gamma}}(z_1, \ldots, z_M)$. For binary classification with $z_m^{(i)} = \widehat{p}_m(y = 1 \mid x^{(i)})$

a common meta-model is logistic regression,

$$\widehat{p}^{(i)}_{\text{stack}} = \sigma\Big(\beta_0 + \sum_{m=1}^{M} \beta_m z^{(i)}_m\Big), \qquad \hat{\beta} = \arg\min_{\beta} \sum_{i=1}^{N} \ell(y^{(i)}, \widehat{p}^{(i)}_{\text{stack}}),$$

fitted on the true labels and on *out-of-fold* base predictions.

Reading the formula

Soft voting is the special case $\beta_m = 1/M$ (up to the sigmoid). Stacking lets the data decide the weights: a large positive β_m gives model m more influence; a small or negative one down-weights an unhelpful or redundant model. The meta-model is kept simple, a linear model or a logistic regression with regularisation, because it is fitted on only M inputs and must not overfit.

Training and prediction are different

Technique: Out-of-fold predictions

Training the meta-model. Split the training set into K folds. For each fold, fit every base model on the other folds and predict the held-out fold. Join the held-out predictions from all folds: every training observation now has a prediction $z^{(i)}_m$ from a model that never saw it. Fit h_γ on these $(z^{(i)}, y^{(i)})$.
Predicting a new observation. Refit each base model on *all* the training data, compute $(z_1, \ldots, z_M)$ for x_{new}, pass them through the fitted meta-model.

Gotcha

Never train the meta-model on *in-sample* base predictions. A deep tree or forest predicts its own training observations almost perfectly, so the meta-model would learn to trust the models that overfit the most, and its own fit would be unrealistically optimistic. This is leakage of training information into the combination rule. Out-of-fold predictions are the cure. They are not the same as OOB predictions (Definition 5.4.4): OOB uses the bootstrap by-product inside a bagged ensemble; out-of-fold uses an explicit fold split and works for any base model.

Summary: Stacking

- A meta-model learns one global combination of base predictions; logistic regression with regularisation is typical.
- Train it on out-of-fold predictions; predict with base models refitted on all data.
- In-sample base predictions leak and favour overfitting models.

5.13 Mixture of Experts

What are we trying to get?

Stacking learns one combination rule for all inputs. We want the architecture where the combination depends on x: a gate decides, for each input, how much to trust each expert, and gate and experts are trained together.

Definition 5.13.1 (Mixture of experts)**.** Experts $p_m(y \mid x)$, $m = 1, \dots, M$, are combined by a *gating model* $g_m(x) \geq 0$ with $\sum_m g_m(x) = 1$:

$$p(y \mid x) = \sum_{m=1}^{M} g_m(x)\, p_m(y \mid x).$$

A linear gate scores expert m by $a_m(x) = \beta_m^\top x + b_m$ (a small network for nonlinear routing) and normalises with a softmax, $g_m(x) = e^{a_m(x)} / \sum_r e^{a_r(x)}$. *Soft routing* uses all experts with their weights; *top-k routing* evaluates only the k experts with the largest scores.

Reading the formula

Compare with stacking's $\hat{y} = h(z_1, \dots, z_M)$: there the weights β_m are the same for every x; here $g_m(x)$ changes with the input, so one expert can own one region of feature space and another a different one. Top-k routing is also a computational device: with M large experts and $k = 2$, only two are evaluated per input (*conditional computation*), which is how very large language and vision models use mixtures of experts.

Technique: Joint training

Minimise the negative log-likelihood of the mixture,

$$-\sum_{i=1}^{N} \log \Big[\sum_{m=1}^{M} g_m(x^{(i)})\, p_m(y^{(i)} \mid x^{(i)}) \Big],$$

jointly over the gate parameters and the expert parameters. Experts improve on the observations that receive high gate weight; the gate learns which expert does well where; repeated joint updates make the experts specialise. The routing regions are not specified in advance.

Gotcha

Expert collapse. An expert that receives more gate weight early learns faster, becomes better, and attracts still more inputs, until one expert handles almost everything and the others are idle. Load-balancing penalties that encourage the gate to spread its weight are used to counter this.

Summary: Mixture of experts

- $p(y \mid x) = \sum_m g_m(x) p_m(y \mid x)$ with softmax gate weights that depend on x.

- Gate and experts trained jointly on the mixture likelihood; soft or top-k routing.
- Differs from stacking by input-dependent weights and joint training; risk of expert collapse.

5.14 Where Ensembles Are Used, and Lab 3

Method	Representative applications	Why it is used
Voting / averaging	medical prediction, sensor fusion, deep-network ensembles	combines different models; more stable predictions
Random forest	intrusion detection, remote sensing, bioinformatics, tabular problems	strong baseline with nonlinearity, interactions, little preprocessing
Gradient-boosted trees	credit scoring, fraud detection, churn, ranking on tabular data	high predictive accuracy from successive corrections
Stacking	data-science competitions, systems combining model families	learns when one model should complement another
Mixture of experts	large language and vision models, multi-domain prediction	specialisation and conditional computation

These are common examples; the appropriate method still depends on the data, the cost and the validation results.

Example 5.14.1 (Lab 3 at a glance)**.** Tree ensembles on the cybersecurity flow data of Labs 1 and 2, with one fixed stratified split and identical rows for every model.

Experiment	Models	Question
Binary classification	bagged trees, random forest	how much does aggregation stabilise the model?
Boosting	gradient boosting, XGBoost	how do rate, depth and rounds interact?
Model comparison	best Lab 2 models and both ensembles	which tree errors are corrected?
Multiclass	random forest, boosting	which attack classes remain difficult?
Port information	with and without destination port	does service information help?
Efficiency	fit time, prediction time, model size	what does higher accuracy cost?

Notes on the original slides

All numbers were rechecked: the variance $0.232\sigma^2$ for $M = 25$, $\rho = 0.2$; the Gini reductions 0.30 and 0.125 of the split example; the MDI credit 0.333 and the normalised MDI 0.75/0.15/0.10; the permutation increases 0.08/0.03/0.01; the boosting round $F_1(7.1) = 20.5$; the derivatives $g = \widehat{p} - y$, $h = \widehat{p}(1 - \widehat{p})$; and the gain formula. All are correct. Points to note when reading the deck:

- **Slide order versus concept order.** The slides introduce bagging (slides 16–22) before decision trees (23–27). This chapter defines trees first (§5.3) because bagging's tree-size hyperparameter and everything in random forests presuppose them.

- **Sign conventions.** The slides use *both* $r = y - \widehat{p}$ (the negative gradient, slides 51, 54) and $g = \widehat{p} - y$ (the gradient, slides 53, 57, 63, where it is written $g_k = -(y_k - \widehat{p}_k)$). They are consistent: $r = -g$. Always ask "gradient or negative gradient?" before reading a formula.
- **Slide 45, Step 5.** Only ID 3 is updated on the slide; the full update is $F_1 = (19, 20.5, 20.5)$ with new residuals $(-9, -0.5, 9.5)$, as in Example 5.7.4.
- **Slide 60, "Variance Target".** The XGBoost structure target is the Newton pseudo-target $r_{\text{XGB}} = -g/h$; "variance" refers to the weight $h = \widehat{p}(1-\widehat{p})$, the Bernoulli variance, not to the target itself.
- **Slide 62, "K independent trees per round".** They are independent only *within* a round and only under the diagonal Hessian approximation (slide 64); the rounds remain sequential.
- **Slide 65.** The three steps are run together with stray "item" markers; the intended list is the one in §5.10.
- **Slide 24.** The λJ penalty on the number of leaves is the same idea as XGBoost's γQ (slide 56), under a different letter; the slides' λ on slide 55 onwards is the L_2 penalty on leaf outputs.
- **Slide 6** appears three times in the PDF (an animation exported as separate pages); nothing is lost.

Chapter takeaways

Summary: Learning by combining models

- **Why:** one flexible fit has high variance; averaging competent models with *different* errors cancels part of it. Var $= \sigma^2(\rho + (1-\rho)/M)$: correlation leaves a floor.
- **Voting:** hard counts classes, soft averages posteriors and needs calibration.
- **Trees:** partitions with one constant per leaf (mean/median/majority); greedy Gini splits; flexible but high-variance.
- **Bagging:** bootstrap samples (63.2% in, 36.8% out); OOB error is free internal validation, not a test set.
- **Random forests:** m_{try} random candidates per split decorrelate trees; tune at forest level; trees until the OOB plateau.
- **Importance:** MDI (training, fast, biased) and permutation (prediction, honest); masking by correlation; never causality.
- **Boosting:** $F_m = F_{m-1} + \eta T_m$; fit the negative gradient; squared error $\Rightarrow$ residuals; M, η, depth tuned jointly with early stopping.
- **Classification:** boost the logit; $g = \widehat{p} - y$, $h = \widehat{p}(1-\widehat{p})$; Newton leaf $\sum(-g)/(\sum h + \lambda)$.
- **XGBoost:** one regularised second-order objective for splits and outputs; $w^* = -G/(H+\lambda)$; gain minus γ per split.
- **Multi-class:** K trees per round, diagonal Hessian separates classes.

- **Stacking** learns a global combination from out-of-fold predictions; **mixture of experts** learns an input-dependent one jointly with the experts.
- **Limits:** shared missing information, identical models, noisy labels, reused validation data, deployment shift.